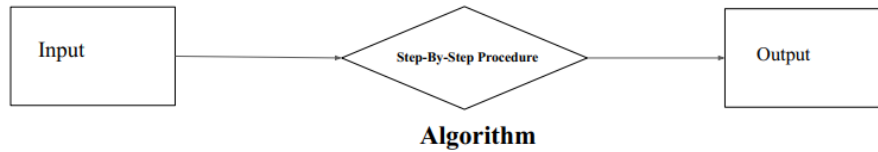


Algorithm

Pdf – 1

1. What is Algorithm?

Ans: An algorithm is a step-by-step procedure or a set of instructions designed to perform a specific task or solve a particular problem.



2. What is the need for algorithms?

1. **Efficiency:** Algorithms help in solving problems in an efficient manner.
2. **Problem-Solving:** Algorithms are a framework for solving problems systematically.
3. **Scalability:** Well-designed algorithms allow systems to handle increasing loads of data.
4. **Standardization:** Algorithms provide a standardized way to solve specific types of problems.
5. **Optimization of Costs:** Algorithms help achieve these goals through optimization techniques.
6. **Complex Problem Solving:** Some problems, especially in fields like cryptography, optimization, and artificial intelligence, are highly complex and require sophisticated algorithms to be solved.

3. What are the Criteria for an algorithm?

1. **Input:** Zero or more quantities are externally supplied.
2. **Output:** At least one quantity is produced.
3. **Definiteness:** Each instruction is clear.
4. **Finiteness:** The algorithm terminates after a finite number of steps.
5. **Effectiveness:** Every instruction must be basic and feasible.

4. Advantages of Algorithms.

1. Easy to Understand
2. Step-by-Step Representation
3. Simplifies Complex Problems
4. Language Independence
5. Easy to Debug and Test
6. Improves Efficiency & Accuracy
7. Optimization

5. What is Complexity Analysis? Why it needs?

Complexity analysis evaluates the efficiency of an algorithm based on its resource consumption.

Why it needs:

1. Ensures efficient resource usage.
2. Helps in comparing algorithms.
3. Critical for scalability with large inputs.

6. Work on Best, Worst and Average Cases.

Best-case: Minimum time for most favorable input.

Worst-case: Maximum time for the least favorable input.

Average-case: Expected time for random inputs.

Pdf – 2

7. Big Oh, O: Asymptotic Upper Bound.

A function $f(n)$ can be represented is the order of $g(n)$ that is $O(g(n))$, if there exists a value of positive integer n as n_0 and a positive constant c such that –

$$f(n) \leq c \cdot g(n) \text{ for } n > n_0 \text{ in all case}$$

Hence, function $g(n)$ is an upper bound for function $f(n)$, as $g(n)$ grows faster than $f(n)$.

8. (11-14) page Find Time and Space Complexity.

Pdf – 4

9. Bubble Sort:

i. Explain Bubble sort with Algorithm Step.

Answer: Hand practices bubble sort.

Algorithm Steps:

1. Start from the first element.
2. Compare the current element with the next element.
3. If the current element is greater than the next element, swap them.
4. Move to the next element and repeat step 2 and 3.
5. Repeat the process for the entire list until no swaps are needed.

Time Complexity: $O(n^2)$

Auxiliary Space: $O(1)$

ii. Advantages and Disadvantages of Bubble sort.

Advantages:

1. Bubble sort is easy to understand and implement.
2. It does not require any additional memory space.
3. It is a stable sorting algorithm

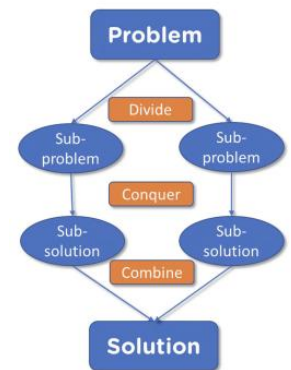
Disadvantages:

1. Inefficient for large datasets.
2. High time complexity compared to other sorting algorithms.

10. Explain Divide and conquer Algorithm with diagram.

Divide-and-conquer recursively solves subproblems. each subproblem must be smaller than the original problem, and each must have a base case. **A divide-and-conquer algorithm has three parts:**

- Divide up the problem into a lot of smaller pieces of the same problem.
- Conquer the subproblems by recursively solving them. Solve the subproblems as base cases if they're small enough.
- To find the solutions to the original problem, combine the solutions of the subproblems.



11. Merge Sort:

i. Explain Merge sort with Algorithm step.

Ans: Hand practices Merge sort.

Algorithm Step:

- Step 1: **Divide:** The primary step is to repeatedly divide the input list into two halves until you reach sub lists containing only one element.
- Step 2: **Conquer (Merge):** After dividing, the algorithm starts merging the single-element lists back together in sorted order.
- Step 3: **Combine:** The merging process continues until all the sub lists are merged into one single sorted list.

ii. Advantages and Disadvantages of Merge Sort.

Advantages:

1. Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
2. Merge sort has a worst-case time complexity of $O(N \log N)$, which means it performs well even on large datasets.
3. The divide-and-conquer approach is straightforward.

Disadvantages:

1. Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
2. Not Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data.
3. Merge Sort is Slower than Quick Sort.

12. Quick Sort:

i. Explain Quick Sort with Algorithm step.

Ans: Hand practices quick sort.

Algorithm Step:

Step1: Choose the pivot (usually first or last element).

Step2: Set left pointer at low, right pointer at high.

Step3: While left $\leq$ right:

- i. Move left right while $\text{arr}[\text{left}] < \text{pivot}$
- ii. Move right left while $\text{arr}[\text{right}] > \text{pivot}$
- iii. If left $<$ right, swap $\text{arr}[\text{left}]$ and $\text{arr}[\text{right}]$.
- iv. Otherwise, return the right pointer as the partition index.

ii. Advantages and Disadvantages of Quick Sort.

Advantages:

1. It is a divide-and-conquer algorithm that makes it easier to solve problems.
2. It is efficient on large data sets.
3. It has a low overhead, as it only requires a small amount of memory to function.

Disadvantages :

1. Worst case is $O(n^2)$ (if poor pivot selection).
2. Not stable (relative order of equal elements may change).

13. Merge Sort VS Quick Sort

Quick Sort	Merge Sort
1.The splitting of a array of elements is in any ratio, not necessarily divided into half.	1.In the merge sort, the array is parted into just 2 halves
2.Worst case complexity $O(n^2)$	2.Worst case complexity $O(n \log n)$
3.Less Additional storage space requirement	3.More Additional storage space requirement
4.Not Stable	4. Stable

Pdf – 3

14. What is Linear Search Algorithm?

Linear search is a simple search algorithm used to find a particular value in a list or an array. The algorithm for linear search is relatively simple.

Linear Search Step:

Step 1: Start at the first element (index 0) of the array.

Step 2: Compare the target value (key) with the value at the current position.

Step 3: If they are the same, return the position of the match.

Step 4: If they are not the same, move to the next element and repeat the comparison.

Step 5: Continue this process until a match is found or the entire array has been checked.

Step 6: If no match is found, print a message saying the element is not in the array and stop the program.

15. Advantages of the Linear Search Algorithm.

Disadvantages of the Linear Search Algorithm.

- ✓ **Simplicity**
- ✓ **No need for sorting**
- ✓ **Handles all data types**
- ✓ **Small dataset**
- ✓ **Low memory usage**

- ✓ **Inefficient for Large Datasets**

16. What is Binary Search Algorithm?

Binary search is a search algorithm used to find the position of a target value within a sorted array. The data structure must be sorted.

Binary Search Step:

1. Find the middle index of the search space.
2. Compare the middle element with the target key
3. Repeat the process on the selected half until the key is found or no elements remain.
4. If the key is not found, return -1 (not found).

17. Advantages of Linear Search Algorithm.

1. Binary search is faster than linear search, especially for large arrays.
2. More efficient than other searching algorithms with a similar time complexity.
3. Binary search is well-suited for searching large datasets that are stored in external memory.

Disadvantages:

1. The array should be sorted